Comparative Analysis of Concurrency Models for JVM-based Backend Services

Dmitrii Prosvirkin
Supervisor: Eugene Zouev

Table of Contents

1 Chapter 1. Introduction 2
1.1 Context 2
1.2 Problem 2
1.3 Object and subject 2
1.4 Properties 2
1.5 Goal and question 3
1.6 Result and scope 3
1.7 Structure 3
2 Chapter 2. Literature Review 3
2.1 The five models 3
2.2 Saturation and where the limit goes 4
2.3 Prior comparisons and the gap 4
3 References 5

1 Chapter 1. Introduction

1.1 Context

A JVM backend serves many requests at once. A request may spend time on the CPU, blocked on another service, or both. The concurrency model is how that work is placed on threads. The model determines which resource is exhausted first — the thread pool, the cap on simultaneous calls to a dependency, or memory (every request in flight holds a stack or a continuation) — and where in the service an explicit limit has to be set.

1.2 Problem

Public comparisons usually take two or three of these models at a time and report throughput, latency, and memory for one or two workloads. They also mix two factors. The first is the scheduler: a bounded pool of platform threads, or virtual threads that leave the carrier thread while blocked. The second is the abstraction: blocking code, Scala `Future`, ZIO fibers, or Kotlin coroutines. A single throughput number from such a mix does not say which resource saturated, and it does not say where the limit should be configured. For Java threads the placement has been studied: Oracle's guide for virtual threads recommends a `Semaphore` around the call to a capped service, and two public experiments, a Swift database test and a Spring Boot load-shedding test, measured where the limit works best. For `Future`, ZIO, and Kotlin coroutines nothing comparable was found (Chapter 2).

1.3 Object and subject

The object of the study is a JVM backend that processes requests in parallel. The subject is how five concurrency models differ on one property list when the request logic is the same.

The models are:

1. A bounded pool of platform threads, with blocking code.
2. Virtual threads (Java 21) and the same blocking code.
3. Scala `Future` on an `ExecutionContext` backed by a thread pool.
4. ZIO fibers on a worker pool.
5. Kotlin coroutines on a dispatcher backed by a thread pool.

Every experiment uses the same logical request: a CPU loop; a wait on an in-process stub of an external service; a request with several outbound calls; cancellation on timeout. The last two parts belong to the composite scenario. The stub has a fixed delay, a cap on simultaneous calls, and a count of occupied slots. The study compares models, not business algorithms.

In all five models the call to the stub blocks. Models 1, 3, 4, and 5 run on a fixed pool of platform threads, so they differ only in the abstraction; model 2 differs in the scheduler. Implementation details are given in the benchmark chapter.

1.4 Properties

The models are compared on five groups of properties.

- **Resources and limits.** Which resource runs out first: the thread pool, the external cap, or the runtime's worker pool. How many OS threads are used. How much overhead remains at the same load, measured as the OS thread count and heap use. How many requests can stay in flight before latency breaks.
- **Behavior under load.** Throughput. Latency at the 95th and 99th percentiles. Overload when offered concurrency exceeds a limit: the queue grows, and new work waits.
- **Cancellation.** Whether cancel reaches the parallel outbound calls. Whether an occupied slot is released. Whether child tasks are tied to the parent request, or must be cancelled by hand.

- **Waiting.** A blocking wait on a pool thread, against an explicit non-blocking wait (a variant outside the main runs). The risk of occupying a pool that was built for asynchronous tasks.
- **Errors and engineering.** What happens when one of several parallel calls fails. Where the parallelism limit is set. What it takes to write the same request in all five models, compared as lines of code and the constructs used; this row is qualitative.

The first three groups are measured. The last two are assessed from the code and the literature.

1.5 Goal and question

The goal is to compare the five models on this list and to show where parallelism should be bounded and how cancellation behaves.

The tasks are:

1. Fix the property list and describe the five models.
2. Build a benchmark with a stub that controls delay, the concurrency cap, and occupied slots.
3. Run CPU-only load, I/O below the stub cap, I/O above the stub cap, a composite request, and cancellation.
4. Identify, for each model, the resource that is exhausted first when the pool size and the stub cap are varied separately.
5. Produce a model-by-property table, recommendations, and the scope of those recommendations.

The research question is: how does the choice of concurrency model change the limits of a JVM backend, and where in the architecture must a bound on parallel work be set?

The working hypothesis is that models 1, 3, 4, and 5 are limited by the pool size, because a blocking call holds a pool thread, and so behave alike up to their own overhead. A virtual thread releases its carrier while it waits, so it should reach the cap of the stub. If `Future`, ZIO, and Kotlin wait without blocking, they should behave like virtual threads; this variant is checked separately. The differences between the models are then expected mainly in overhead and in cancellation.

1.6 Result and scope

The result is the property list, the measurements, the table, and the recommendations.

The work stays on one JVM. The dependency is a stub, not a network service. The load is closed-loop: a fixed number of workers, each sending the next request when the previous one ends; an open-loop variant with a fixed arrival rate is discussed in the benchmark chapter. Distributed actors, reactive streams (Project Reactor), retry policy, adaptive concurrency limits, cache-level effects, multi-node runs, production traces, and a survey of HTTP or JDBC clients are out of scope. Pinning of a virtual thread inside `synchronized` or native code is also out of scope.

1.7 Structure

Chapter 2 reviews the five models and the literature on saturation, and states the gap. The following chapter treats limits, cancellation, and overload. The benchmark chapter fixes the shared request, the stub, the scenarios, and the metrics, and states the threats to validity. The last chapter gives the comparison table and the recommendations.

---

2 Chapter 2. Literature Review

2.1 The five models

Goetz et al. treat threads and thread pools as the basic tools of concurrent Java [1]. A fixed pool is both a scheduler and a limit: when every worker is blocked in I/O, further tasks queue and latency grows

even if the dependency could accept more calls. The book does not treat a dependency's own cap as a separate resource.

JEP 444 defines a virtual thread as a JVM thread that unmounts from its carrier during blocking I/O [2]. Many blocked requests then occupy few OS threads, so one would expect the dependency cap or memory, not the pool, to be exhausted first; the JEP does not measure this. The Java 21 guide puts the limit in a `Semaphore` around the call to the capped service, and says that virtual threads should not be pooled for this purpose [3]. `StructuredTaskScope` cancels unfinished subtasks when the scope ends, but it is still a preview API [4], so the Java baseline in this thesis is plain blocking code, with cancellation by thread interruption. These texts state the rule. They do not measure what happens at the cap, and they do not cover other models.

In Scala, a `Future` runs on the pool chosen by its `ExecutionContext`, does not limit how many tasks are submitted (the pool size limits only how many run at once), and a blocking call inside it holds a pool thread [5]. ZIO schedules fibers on a worker pool, has a separate path for blocking work, and treats interruption as part of the fiber [6]. In Kotlin, a suspend function frees its thread at a suspension point, dispatchers place coroutines on pools, and `coroutineScope` cancels the children together with the parent [7], a rule introduced by Elizarov in 2018 [8]. A blocking call on a limited dispatcher still holds a thread. The documentation of these three describes the programming model. It does not separate the worker-pool size from an external cap, and it does not say whether a slot is released after cancellation.

2.2 Saturation and where the limit goes

Kleppmann describes how queueing delay and head-of-line blocking dominate the high percentiles of response time [9]. Dean and Barroso show that for a request that waits on several components these percentiles decide what the user sees [10]. Little's law links the average number of requests in a system to their arrival rate and the time each spends there, $L = \lambda W$ [11]. For a stub with N slots and delay D, every call holds a slot for D, so N slots serve at most $\frac{N}{D}$ calls per unit of time, and reach that rate only if callers keep all slots busy. The SRE book lists saturation among the signals to watch [12]. Other sources say where a limit goes. The HikariCP note argues that once the database is the bottleneck, a larger connection pool adds contention, not throughput [13]. Nygard lists bulkheads, timeouts, back pressure, and load shedding as patterns that bound work [14]. Netflix computes the limit of in-flight requests at run time and publishes a Java library [15]. All of them show that a limit must exist and why. None says which of the five models reaches the limit of the stub and which is stopped earlier by its own pool, and none checks whether a cancelled request returns its slot.

Two reports measure the placement on Java threads. Swift describes a database test in which a fixed pool of 40 platform threads limited concurrency by accident. Replacing it with virtual threads removed the limit and raised tail latency by an order of magnitude; a `Semaphore` with 20 permits brought the 99.999th percentile from over 10 s to 200 ms [16]. Popov runs Spring Boot with virtual threads against a stub with a 200 ms delay and a cap of 1000 calls, and compares a limit at the server, in the application, and at the stub only. At 3x of capacity the server-level limit keeps 88% of peak goodput, the application-level limit about 70%, and with only the stub limit goodput collapses between 1.25x and 1.5x [17]. Popov lists as open that a client timeout does not stop the work already running in the service. Both reports cover Java threads only. For virtual threads the placement is therefore partly answered; for `Future`, ZIO, and Kotlin on a blocking call it is not.

2.3 Prior comparisons and the gap

Comparisons of these models exist, but only in pairs and triples. Several sources are theses, blog posts, or mailing-list reports, so they are used as reported observations. For several academic items only the abstract was read.

Java threads. Lasic et al. compare platform and virtual threads in database-backed servers [18]. Pandita finds equal throughput on CPU-bound work and higher throughput for virtual threads on I/O-bound work [19]. Gustafsson and Nederlund Persson add reactive streams and limited and unlimited platform pools; virtual threads give lower latency and reactive streams lower memory use [20]. Hu is the closest. In SPECjbb2015 on JDK 25 virtual threads have lower peak throughput, and bounding their number raises the throughput metrics by about 16–20% [21]. This supports the claim that an explicit bound matters, but the study covers only the two thread types and no capped dependency.

Virtual threads and Kotlin coroutines. Beronić et al. compare platform threads, virtual threads, and coroutines on structured-concurrency constructs [22]. Grabowiec et al. find coroutines on the IO dispatcher about 16% faster than a thread pool on a REST workload, with virtual threads behind at higher iteration counts [23]. Mohammed compares virtual threads and coroutines on four JVMs: virtual threads scale better on I/O, coroutines have lower overhead on CPU work [24].

Effect systems. Spiewak shows that the ZIO and Cats Effect microbenchmarks each stress a narrow artificial workload [25]. VirtusLab compares `Future`, Cats Effect, ZIO, Kyo, Gears, and Ox on a crawler task with a parallelism limit and cancellation on failure, as a code comparison without measurement [26]. A blog post tabulates platform threads, virtual threads, ZIO fibres, and Kotlin coroutines, including interruption, with approximate figures [27]. It is the nearest published property table, but it has no `Future`, no dependency cap, and no measured cancellation. Queries in OpenAlex, Crossref, and arXiv returned no peer-reviewed comparison of ZIO or Cats Effect with other concurrency models.

Cancellation. Ward's 2024 talk notes compare racing, loser cancellation, and resource closing in ZIO, Kyo, Ox, Java with Loom, and Rust, and record that loser cancellation was not validated in one scenario [28]. AsyncLeakBench collects 902 fixed asynchronous resource leaks in Java projects, with categories triggered by cancellation and timeouts, and reports that detectors handle those triggers poorly [29]. This shows that a resource can stay held after cancellation, but not for a slot in a concurrency limit.

Benchmark suites. Renaissance, the peer-reviewed suite of concurrent JVM workloads, was built to evaluate compilers and VMs [30]. Its workloads include Scala `Future`, Akka, Rx, Finagle, and Ktor, and it has no capped dependency, no request-level tail latency, and no cancellation scenario [31]. It also advises that a few repetitions give only a quick look and a thorough evaluation needs many more [31] the benchmark chapter follows this.

What is not covered. In the sources found, no single study combines:

1. all five models of this thesis, with `Future` and ZIO next to Java threads and Kotlin;
2. the same request in every model, with a call to an external dependency that has its own cap;
3. the dependency cap and the pool size varied separately, to identify the first exhausted resource;
4. measured cancellation, including whether a slot in the cap is released when a fan-out is cancelled.

---

3 References

1. Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., & Lea, D. (2006). *Java Concurrency in Practice*. Addison-Wesley.
2. OpenJDK. (2023). *JEP 444: Virtual Threads*. https://openjdk.org/jeps/444
3. Oracle Corporation. *Virtual Threads*. Java SE 21. https://docs.oracle.com/en/java/javase/21/core/virtual-threads.html
4. OpenJDK. (2025). *JEP 525: Structured Concurrency (Preview)*. https://openjdk.org/jeps/525

5. Odersky, M., Spoon, L., Venners, B., & Sommers, F. (2021). *Programming in Scala* (5th ed.). Artima Press.

6. ZIO Contributors. *ZIO Documentation*. https://zio.dev/reference/core/runtime/

7. JetBrains. *Coroutines guide*. Kotlin. https://kotlinlang.org/docs/coroutines-guide.html

8. Elizarov, R. (2018, September 12). Structured concurrency. *Medium*. https://elizarov.medium.com/structured-concurrency-722d765aa952

9. Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.

10. Dean, J., & Barroso, L. A. (2013). The tail at scale. *Communications of the ACM, 56* (2), 74–80. https://doi.org/10.1145/2408776.2408794

11. Little, J. D. C. (1961). A proof for the queuing formula: $L = \lambda W$. *Operations Research, 9* (3), 383–387. https://doi.org/10.1287/opre.9.3.383

12. Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (Eds.). (2016). *Site Reliability Engineering*. O'Reilly Media.

13. HikariCP. *About Pool Sizing*. https://github.com/brettwooldridge/HikariCP/wiki/About-Pool-Sizing

14. Nygard, M. T. (2018). *Release It! Design and Deploy Production-Ready Software* (2nd ed.). Pragmatic Bookshelf. ISBN 978-1-68050-239-8.

15. Landau, E., Thurston, W., & Bozarth, T. (2018, March 23). Performance under load: Adaptive concurrency limits @ Netflix. *Netflix Technology Blog*. https://netflixtechblog.medium.com/performance-under-load-3e6fa9a60581 (library: https://github.com/Netflix/concurrency-limits)

16. Swift, M. (2024, July 6). Experience using virtual threads in EA 23-loom+4-102. *loom-dev* mailing list, OpenJDK. https://mail.openjdk.org/pipermail/loom-dev/2024-July/006881.html

17. Popov, A. (2026, September 4). Spring Boot under overload: Verify load shedding strategies. *Popov R&D*. https://blog.popov-rnd.com/posts/spring-boot-under-overload/

18. Lasic, L., Beronić, D., Mihaljević, B., & Radovan, A. (2024). Assessing the efficiency of Java virtual threads in database-driven server applications. In *Proc. 47th MIPRO ICT and Electronics Convention* (pp. 2045–2050). IEEE. https://doi.org/10.1109/MIPRO60963.2024.10569754

19. Pandita, V. (2024). *Benchmarking the performance of Java virtual threads in high-throughput workloads* (MSc research project). National College of Ireland. https://norma.ncirl.ie/8134/1/visheshpandita.pdf

20. Gustafsson, E., & Nederlund Persson, O. (2024). *Comparison of concurrency technologies in Java: Structured testing in a high load environment* (Master's thesis, LU-CS-EX 2024-31). Lund University. https://lup.lub.lu.se/luur/download?fileOId=9166687&func=downloadFile&recordOId=9166685

21. Hu, Z. (2026). *Platform threads versus virtual threads in SPECjbb2015: An empirical study on OpenJDK 25* (Master's degree project). Uppsala University. https://uu.diva-portal.org/smash/get/diva2:2084789/FULLTEXT01.pdf

22. Beronić, D., Modrić, L., Mihaljević, B., & Radovan, A. (2022). Comparison of structured concurrency constructs in Java and Kotlin – virtual threads and coroutines. In *Proc. 45th MIPRO ICT and Electronics Convention* (pp. 1466–1471). IEEE. https://doi.org/10.23919/MIPRO55190.2022.9803765

23. Grabowiec, M., Wiktor, S., & Smółka, J. (2024). Performance analysis of coroutines and other concurrency techniques in Kotlin language for I/O operations. *Journal of Computer Sciences Institute*. https://doi.org/10.35784/jcsi.6353

24. Mohammed, I. (2026). *A comparative study of virtual threads and coroutines in Kotlin across different JVM implementations* (degree project, in Swedish). DiVA, urn:nbn:se:kth:diva-388719. http://urn.kb.se/resolve?urn=urn%3Anbn%3Ase%3Akth%3Adiva-388719

25. Spiewak, D. (2021, July 30). *Understanding comparative benchmarks* (Cats Effect 3 and ZIO 2). https://gist.github.com/djspiewak/f4cfc08e0827088f17032e0e9099d292

26. VirtusLab. (2026, April 17). Comparing effect systems in Scala: The problem and Future. https://virtuslab.com/blog/scala/comparing-effect-systems-in-scala

27. sps23. (2026, May 25; updated 2026, August 29). ZIO fibres vs Java virtual threads vs Kotlin coroutines. *Java for Scala Developers*. https://javaforscaladevs.com/blog/2026/05/25/zio-fibres-vs-virtual-threads-vs-coroutines/

28. Ward, J. (2024). *Comparing approaches to structured concurrency* (LambdaConf 2024 talk notes, EasyRacer project). https://jamesward.github.io/easyracer/lambdaconf_2024.html

29. Kim, J., Heo, J., Choi, D., & Lee, E. (2026). AsyncLeakBench: A curated benchmark of asynchronous resource leaks in open-source Java projects. *Proceedings of the ACM on Software Engineering, 3* (ISSTA), Article ISSTA197. https://doi.org/10.1145/3832288

30. Prokopec, A., Rosà, A., Leopoldseder, D., Duboscq, G., Tůma, P., Studener, M., Bulej, L., Zheng, Y., Villazón, A., Simon, D., Würthinger, T., & Binder, W. (2019). Renaissance: Benchmarking suite for parallel applications on the JVM. In *Proc. 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '19)* (pp. 31–47). ACM. https://doi.org/10.1145/3314221.3314637

31. Renaissance Benchmark Suite. *README*, version 0.16.2. https://github.com/renaissance-benchmarks/renaissance